

GENERALIZED STAR HEIGHT FOR LEAN EXPERTS

RYUYA HORA AND THE ZMC GENERALIZED STAR-HEIGHT WORKING GROUP

ABSTRACT. This primer introduces the generalized star-height project to Lean users who need enough formal-language theory to design a durable formalization. It explains the mathematical quantifiers, the distinction between syntax height and semantic language height, the intended trusted computing base, the repository architecture, and the principal representation choices. It then gives Lean-facing interfaces for languages, generalized regular expressions, deterministic automata, recognition, syntactic monoids, finite-group milestones, executable certificates, and eventual lower-bound invariants. The emphasis is on specification engineering: theorem statements must remain useful while the research direction changes, computational search must produce independently checkable certificates, and open mathematical claims must never enter the environment as axioms.

CONTENTS

1. The mathematical target and its logical shape	2
1.1. The quantifier trap	2
2. Repository policy before mathematics	2
2.1. Status labels	2
2.2. No research axioms	2
2.3. Agent instructions as versioned source	2
3. Core representations	3
3.1. Words and languages	3
3.2. Generalized regular expressions	3
3.3. Why complement is semantic, not finite	3
4. Automata and recognition	3
4.1. A minimal DFA structure	3
4.2. Recognition by a monoid	4
5. Syntactic monoids without premature abstraction	4
5.1. Recommended construction order	4
5.2. Aperiodicity interface	4
6. Finite groups and the milestone ladder	4
6.1. Do not formalize a presentation without a model theorem	5
7. The certificate boundary	5
7.1. Why certificates are central	5
7.2. Lean soundness target	5
7.3. Recommended staged trust model	5
8. How to formalize upper-bound constructions	6
8.1. A transport-theorem pattern	6
8.2. Proof-producing constructors	6
9. Lower bounds need a constructor calculus	6
10. Testing strategy	6
10.1. Three layers	6
10.2. Negative tests	7
10.3. Continuous integration	7
11. A task protocol for coding agents	7
11.1. Suggested roles	7
12. Workshop exercises	7
13. Release gates	8

Date: 22 July 2026.

2020 Mathematics Subject Classification. 68V15, 03B35, 68Q45, 20M35.

Key words and phrases. Lean, mathlib, generalized star height, regular languages, finite monoids, certificates.

14. Reading guide	8
Acknowledgement	8
References	8

1. THE MATHEMATICAL TARGET AND ITS LOGICAL SHAPE

Let A be a finite alphabet and let A^* be the free monoid of finite words. A generalized regular expression is built from empty language, empty word, letters, union, concatenation, complement relative to A^* , and Kleene star. Its syntax-tree star height is the largest number of nested star constructors. The generalized star height of a language L is the minimum syntax height over all expressions denoting L .

The central open conjecture is

$$\forall A \text{ finite } \forall L \subseteq A^*, \quad L \text{ regular} \implies \text{gsh}(L) \leq 1.$$

No generalized-height-greater-than-one regular language is known in the surveyed literature [PST92; Bou17; PZ17]. A Lean project cannot hide this status. It should formalize the statement and the surrounding established theory without declaring the conjecture as an axiom.

1.1. The quantifier trap. The displayed expression for a language is not its minimal expression. An explicit height-one expression proves an upper bound:

$$\exists r, \quad \llbracket r \rrbracket = L \quad \text{and} \quad \text{sh}(r) \leq 1.$$

A lower bound has the opposite logical form:

$$\forall r, \quad \llbracket r \rrbracket = L \implies \text{sh}(r) > 1.$$

This distinction determines the formal architecture. Upper-bound work can use compact certificates. Lower-bound work needs an induction principle over every height-one expression, or a theorem characterizing the full height-one class.

Warning 1.1.1. A bounded synthesis run that finds no expression is evidence about that run, not a theorem about generalized star height. The formal output of a failed search is a finite exhaustion statement with explicit bounds.

2. REPOSITORY POLICY BEFORE MATHEMATICS

The repository pins Lean and mathlib to the same release tag in `lean-toolchain` and `lakefile.toml`. This follows the official dependency model rather than tracking an unpinned moving branch [Lea26; mat26b]. The initial release is a research scaffold, not a claim of full formalization.

2.1. Status labels. Every major claim is assigned one of the labels PROVED, CITED, COMPUTED, CONJECTURAL, SPECULATIVE, REFUTED, or UNREVIEWED. The ledgers `CLAIMS_LEDGER.md` and `PROOF_OBLIGATIONS.md` connect prose to code and sources. A declaration containing `sorry` remains UNREVIEWED; it is not silently promoted by successful compilation.

2.2. No research axioms. Open conjectures should be definitions of propositions:

```
def GeneralizedHeightOneConjecture : Prop :=
  forall (A : Type) [Fintype A] [DecidableEq A],
    HeightOneCollapse A
```

This allows other files to state conditional theorems without contaminating the kernel. If a temporary interface is unavoidable, use a theorem with `sorry`, tag it with a unique blueprint identifier, and register the hole. Never use `axiom` to make a speculative bridge available to automation.

2.3. Agent instructions as versioned source. The files `AGENTS.md`, `CLAUDE.md`, and `.claude/rules/lean.md` specify commands, file ownership, proof-status rules, and stopping conditions. Repository-local instructions are preferable to repeatedly pasting a large prompt and match current recommendations for coding-agent context files [Ope26a; Ant26]. An agent receives one task packet, one verification command, and one bounded output contract.

3. CORE REPRESENTATIONS

3.1. Words and languages. The bootstrap uses lists for words and sets for languages:

```
abbrev Word (A : Type u) := List A
abbrev Language (A : Type u) := Set (Word A)
```

The free-monoid multiplication is list append. This is deliberately concrete. It aligns with executable automata, has mature list induction, and keeps the alphabet parameter visible in complement.

Language concatenation is relational:

```
def Language.concat (L K : Language A) : Language A :=
  {w | exists u in L, exists v in K, u ++ v = w}
```

Kleene star is the union of finite powers. The first formal lemmas should normalize membership rather than immediately seek a large algebraic API. Essential tests are

$$\emptyset^* = \{\varepsilon\}, \quad \{\varepsilon\}L = L, \quad (LK)H = L(KH), \quad L \subseteq L^*.$$

Remark 3.1.1. A more abstract free-monoid representation may eventually support transport across alphabets. It should be introduced only after the list-based semantic kernel is stable, with an explicit equivalence rather than a wholesale rewrite.

3.2. Generalized regular expressions. The syntax is an inductive type:

```
inductive GRegex (A : Type u) where
| zero
| epsilon
| atom : A -> GRegex A
| union : GRegex A -> GRegex A -> GRegex A
| concat : GRegex A -> GRegex A -> GRegex A
| compl : GRegex A -> GRegex A
| star : GRegex A -> GRegex A
```

Its denotation is compositional. Syntax height is computable by recursion. Semantic height is represented first by an upper-bound predicate:

```
def HasHeightAtMost (L : Language A) (n : Nat) : Prop :=
  exists r : GRegex A,
    GRegex.denote r = L /\ GRegex.starHeight r <= n
```

This avoids defining a partial minimum on arbitrary languages. Later, regularity can provide nonemptiness of the set of admissible heights, and well-ordering of natural numbers can produce a least height.

3.3. Why complement is semantic, not finite. For a finite alphabet, the universe is still the infinite type `List A`. Complement cannot be implemented by enumerating words. At the syntax and theorem levels, it is ordinary set complement. At the certificate-checker level, complement is compiled by completing a finite automaton and swapping accepting states. Confusing these layers is a common source of unsound code.

4. AUTOMATA AND RECOGNITION

4.1. A minimal DFA structure. The bootstrap separates the mathematical structure from finiteness assumptions:

```
structure DFA (A : Type u) (Q : Type v) where
  step : Q -> A -> Q
  start : Q
  accept : Set Q
```

The run function recurses on words. Algorithms add `Fintype`, `DecidableEq`, and decidability of acceptance only when needed. This prevents every semantic lemma from carrying computational instances.

The first important theorem is

```
theorem run_append (M : DFA A Q) (q : Q) (u v : List A) :
  M.run q (u ++ v) = M.run (M.run q u) v
```

It is the bridge between concatenation of words and composition of transitions. Many later recognition proofs should reduce to this lemma.

4.2. Recognition by a monoid. Recognition data is

```
structure Recognition (A : Type u) (M : Type v) [Monoid M] where
  morphism : List A ->* M
  accepting : Set M
```

with language the inverse image of the accepting subset. The morphism need not be surjective. Theorems that require a generated image should explicitly replace M by the range or assume surjectivity.

For a finite group G , the workshop property is

```
def HeightOneForGroup (G : Type v) [Group G] : Prop :=
  forall (A : Type u) [Fintype A] [DecidableEq A],
    forall R : Recognition A G,
      HasHeightAtMost R.language 1
```

This is stronger than proving a statement for one generating tuple or one accepting subset. It is still only the group-recognized subclass of regular languages.

5. SYNTACTIC MONOIDS WITHOUT PREMATURE ABSTRACTION

For a language L , define the two-sided contextual relation

$$u \equiv_L v \iff \forall x, y, \quad xuy \in L \iff xvy \in L.$$

The bootstrap proves reflexivity, symmetry, transitivity, and compatibility with append. The quotient carrier is a `Quotient`. The monoid instance is intentionally isolated as a registered proof obligation.

5.1. Recommended construction order.

- (1) Define the setoid and prove two-sided append congruence.
- (2) Build multiplication on the quotient with `Quotient.map` or the appropriate quotient API.
- (3) Prove monoid laws by quotient induction.
- (4) Define the canonical morphism $A^* \rightarrow \text{Syn}(L)$.
- (5) Prove that membership in L is constant on classes.
- (6) State recognition of L by its syntactic monoid.
- (7) Only then formalize the divisor/minimality property.

A generic quotient-monoid framework may already exist in `mathlib`. An API-scout agent should locate it and produce a minimal compiling example before the project commits to a custom instance. The criterion is not line count alone: the chosen representation should make later finite computations and recognition transport manageable.

5.2. Aperiodicity interface. A finite monoid is aperiodic when each element has an eventually stable power. Schutzenberger’s theorem identifies star-free regular languages with aperiodic syntactic monoids [Sch65; MP71]. This theorem is a substantial formalization target, not a one-line wrapper. The interface file may state

```
theorem schutzenberger_interface
  (L : Language A) (hreg : IsRegular L) :
    IsStarFree L <-> HasAperiodicSyntacticMonoid L := by
  sorry
```

but downstream files must not treat the hole as established research output.

6. FINITE GROUPS AND THE MILESTONE LADDER

The finite-group program should not begin with A_5 . The published counting program identifies A_4 and the dicyclic group of order twelve as the first unresolved order-twelve cases in that line of attack [Bou17; BR16]. The repository therefore names separate milestones for A_4 , Dic_3 , and A_5 .

Mathlib represents the alternating group as a subgroup of a permutation group and provides a simplicity theorem for A_5 [mat26a]. The bootstrap uses

```
abbrev A5 := alternatingGroup (Fin 5)

theorem a5_isSimple : IsSimpleGroup A5 := by
  simp [A5] using
    (alternatingGroup.isSimpleGroup (α := Fin 5) (by decide))
```

The exact binder spelling must be checked against the pinned mathlib release; the official documentation is the source of truth. A compile failure here is an API mismatch, not a mathematical problem.

6.1. Do not formalize a presentation without a model theorem. For Dic_3 , relations alone do not identify a unique finite group. A useful concrete model must include:

- a carrier with decidable equality and finiteness;
- the group operation;
- distinguished generators satisfying the relations;
- generation by those elements;
- a proof that the cardinality is twelve;
- an equivalence to any alternate permutation or semidirect-product model used elsewhere.

This is a good finite, reviewable Lean task for a group theorist and a formalizer working together.

7. THE CERTIFICATE BOUNDARY

7.1. Why certificates are central. Expression discovery may use Python, SAT/SMT, stochastic search, or an LLM. None of these should be trusted. A candidate is accepted only after a small checker verifies:

- (1) the claimed syntax height;
- (2) exact semantic equivalence to the target automaton;
- (3) optionally, provenance and resource bounds.

The bootstrap Python checker compiles generalized expressions to complete automata, minimizes them, and returns a shortest distinguishing word when equivalence fails. The generated JSON is a portable certificate, not a proof object yet.

7.2. Lean soundness target. A Lean certificate structure can mirror the semantic content:

```
structure RegexCertificate (A : Type u) (Q : Type v) where
  expression : GRegex A
  claimedHeight : Nat
  target : DFA A Q
```

The final verified checker should have a theorem of the form

```
theorem checker_sound
  [Fintype A] [DecidableEq A]
  [Fintype Q] [DecidableEq Q]
  (C : RegexCertificate A Q)
  (h : check C = true) :
  HasHeightAtMost C.target.language C.claimedHeight
```

The proof may factor through a verified automata-equivalence procedure or through a certificate containing a bisimulation/equivalence relation. The second approach can reduce trusted executable code at the cost of larger certificates.

7.3. Recommended staged trust model.

Stage	Accepted evidence
0	Python tests and human inspection; useful for exploration only.
1	Independent Python checker plus reproducible run manifest.
2	Lean parser for the certificate and a theorem reducing validity to finite checks.
3	Verified Boolean checker with a kernel-checked soundness theorem.
4	Optional extraction or native execution with equivalence to the verified function.

Research can proceed at Stage 1 while formalization develops, provided documents label the trust level accurately.

8. HOW TO FORMALIZE UPPER-BOUND CONSTRUCTIONS

Published proofs often manipulate inverse morphisms, Boolean combinations, products, substitutions, and finite unions of counting languages. Formalize the closure theorem actually used by the proof rather than all of language theory.

8.1. A transport-theorem pattern. For a letter map $f : A \rightarrow B$, let $f^* : A^* \rightarrow B^*$ be the induced monoid morphism. A reusable target is

```
theorem height_le_of_comap_letterMap
  (f : A -> B) (L : Language B) :
  HasHeightAtMost L n ->
  HasHeightAtMost ((FreeMonoid.map f) -1' L) n
```

The exact implementation may use list mapping rather than an abstract free monoid. The proof should construct a translated expression and establish denotational correctness by induction.

Analogous theorems are needed for Boolean operations and concatenation. For star, the syntax-height theorem must record the increase:

$$\text{HasHeightAtMost}(L, n) \implies \text{HasHeightAtMost}(L^*, n + 1).$$

A stronger theorem with no increase requires a special hypothesis and is precisely the kind of mathematical content the project seeks.

8.2. Proof-producing constructors. Instead of proving only existential closure, define expression constructors and prove their specifications. For example:

```
def witnessUnion (r s : GRegex A) : GRegex A :=
  .union r s

theorem denote_witnessUnion ...

theorem height_witnessUnion ...
```

This creates a small proof-producing DSL. AI agents can synthesize terms in the DSL while Lean verifies semantics and bounds.

9. LOWER BOUNDS NEED A CONSTRUCTOR CALCULUS

A future obstruction should be formalized as a predicate or invariant on languages together with closure lemmas for every height-one constructor. A schematic interface is

```
structure HeightOneInvariant (A : Type u) where
  Holds : Language A -> Prop
  starFree : forall L, IsStarFree L -> Holds L
  union : forall L K, Holds L -> Holds K -> Holds (L union K)
  compl : forall L, Holds L -> Holds L.compl
  concat : forall L K, Holds L -> Holds K -> Holds (L.concat K)
  star_of_starFree : forall L, IsStarFree L -> Holds L.star
```

Then structural induction over a normalized height-one expression yields that every height-one language satisfies **Holds**. A candidate language violating it gives a lower bound. The hard mathematical work is usually the concatenation field; formalization exposes this immediately.

Cohomology becomes relevant only after there is a concrete candidate for **Holds** and proofs, or counterexamples, for these fields. Formalizing a spectral sequence before defining the word-to-cochain map is not an efficient dependency order.

10. TESTING STRATEGY

10.1. Three layers.

- (1) *Unit tests for definitions.* Empty alphabet, empty word, complement universe, run append, and syntax height.
- (2) *Property tests outside Lean.* Enumerate small words and compare independent semantics for random expressions and automata.

(3) *Kernel theorems.* Soundness of executable checks and transport constructors.

The external tests should generate counterexamples as concrete words. A failing equality with no witness is much harder to debug across disciplines.

10.2. Negative tests. Preserve intentionally invalid certificates and ambiguous decompositions. Tests should confirm that the checker rejects them and returns the expected shortest witness. In open-problem work, a durable refutation of a tempting lemma is a valuable artifact.

10.3. Continuous integration. The CI order should be cheap-to-expensive:

- (1) Markdown/ledger lint;
- (2) Python unit tests and example certificates;
- (3) `lake build`;
- (4) documentation build;
- (5) optional bounded experimental jobs on manual dispatch.

Do not place long synthesis runs in pull-request CI. Store their manifests and replay commands instead.

11. A TASK PROTOCOL FOR CODING AGENTS

A coding task packet contains:

- (1) one mathematical statement with all quantifiers;
- (2) permitted files;
- (3) required imports and forbidden new axioms;
- (4) one command that certifies completion;
- (5) an explicit non-goal list;
- (6) a stopping rule and token/time budget;
- (7) required report: changed files, theorem names, remaining gaps, and exact errors.

This reflects lessons from public long-horizon mathematical prompting: decompose the problem, ask for falsifiable intermediate artifacts, and separate solution generation from adversarial review [Ope26b; Lou26]. A long run should start only after a short scout has confirmed the relevant API and a human has accepted the theorem statement.

11.1. Suggested roles.

- *API scout*: finds existing mathlib structures and makes a ten-line compiling example.
- *definition engineer*: owns semantics and avoids duplicate abstractions.
- *proof implementer*: proves one registered obligation.
- *certificate engineer*: connects external data to the verified checker.
- *adversarial reviewer*: searches for missing hypotheses and vacuity.
- *integration owner*: rebases, runs the full checks, and updates ledgers.

The agent proposing a theorem should not be the only reviewer of its specification.

12. WORKSHOP EXERCISES

Exercise 12.0.1 (Semantic versus syntactic height). Prove that `HasHeightAtMost L n` is monotone in n . Then prove closure under complement without increasing the bound. Do not unfold more definitions than necessary.

Exercise 12.0.2 (Language monoid laws). Prove associativity of language concatenation and the left and right identity laws for the epsilon language. Record which simp lemmas make later proofs shorter and which create loops.

Exercise 12.0.3 (Expression translation). Given $f : A \rightarrow B$, define a map from expressions over A to expressions over B and prove its denotation theorem. State carefully whether the result concerns direct images or inverse images of languages.

Exercise 12.0.4 (Syntactic quotient). Complete the monoid instance on the syntactic quotient. Add a test theorem showing that the quotient map preserves concatenation. Update the corresponding blueprint identifier and proof-obligation ledger.

Exercise 12.0.5 (Finite equivalence certificate). Design a certificate for equivalence of two complete DFAs, such as a relation containing the initial pair, closed under every letter transition, and respecting acceptance. Prove that a valid relation implies equality of accepted languages.

Exercise 12.0.6 (A_5 API map). Using the pinned mathlib documentation, produce a file that proves finiteness, cardinality 60, noncommutativity, and simplicity of the chosen A_5 model. Every theorem should cite the imported declaration it wraps; do not reprove group theory unnecessarily.

Exercise 12.0.7 (No-search lower bound). Write the Lean statement of a hypothetical invariant theorem sufficient to prove a language has height greater than one. Include every closure field needed for union, complement, concatenation, and a star on a star-free operand. This is a specification exercise, not a request to invent the invariant.

13. RELEASE GATES

A formalization milestone is released only when:

- (1) `lake build` succeeds on a clean checkout with the pinned toolchain;
- (2) all remaining `sorry` declarations are listed by identifier;
- (3) executable certificates pass both positive and negative tests;
- (4) prose claims link to theorem names, exact citations, or run manifests;
- (5) no source-derived theorem has silently changed hypotheses;
- (6) a formal-language theorist approves semantic definitions;
- (7) a Lean reviewer approves the trusted-boundary argument.

The ambitious success state is a Lean-checked new theorem. A useful intermediate state is smaller: a reusable formal kernel, a verified certificate checker, exact formal statements of the A_4 , Dic_3 , and A_5 targets, and a ledger of failed lemmas that prevents repeated work.

14. READING GUIDE

For the mathematics, begin with Pin–Straubing–Therien and Bourne–Ruskuc, then use the separate formal-language and number-theory primers [PST92; BR16; Bou17]. For Lean, consult the Lean reference and current mathlib documentation rather than relying on remembered APIs [MU21; The20; AMK24]. The repository’s `AGENTS.md` and prompt protocol explain the local engineering conventions.

Acknowledgement. This primer uses the package, theorem-environment, bibliography, and macro organization of the supplied Ryuya Hora `FirstVersion.tex` template, with portable-font and workshop-specific additions.

REFERENCES

- [Ant26] Anthropic. *Best Practices for Claude Code*. Accessed 22 July 2026. 2026. URL: <https://code.claude.com/docs/en/best-practices>.
- [AMK24] Jeremy Avigad, Leonardo de Moura, and Soonho Kong. *Mathematics in Lean*. Lean Community, 2024. URL: https://leanprover-community.github.io/mathematics_in_lean/.
- [Bou17] Thomas Bourne. “Counting Subwords and Other Results Related to the Generalised Star-Height Problem for Regular Languages”. PhD thesis. University of St Andrews, 2017. URL: <https://hdl.handle.net/10023/12024>.
- [BR16] Thomas Bourne and Nik Ruškuc. “On the Star-Height of Subword Counting Languages and Their Relationship to Rees Zero-Matrix Semigroups”. *Theoretical Computer Science* 653 (2016), pp. 87–96. DOI: [10.1016/j.tcs.2016.09.024](https://doi.org/10.1016/j.tcs.2016.09.024). URL: <https://hdl.handle.net/10023/11811>.
- [Lea26] Lean FRO. *Lean Release Notes*. Accessed 22 July 2026. 2026. URL: <https://lean-lang.org/doc/reference/latest/releases/>.
- [Lou26] Aaron Lou. *Jacobian Conjecture Prompt*. Publicly shared multi-agent prompt, accessed 22 July 2026. 2026. URL: https://aaronlou.com/jacobian_counterexample_prompt.pdf.
- [mat26a] mathlib Community. *Mathlib.GroupTheory.SpecificGroups.Alternating and .Simple*. Accessed 22 July 2026. 2026. URL: https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/SpecificGroups/Alternating.html.
- [mat26b] mathlib Community. *Using mathlib4 as a Dependency*. Accessed 22 July 2026. 2026. URL: <https://github.com/leanprover-community/mathlib4/wiki/Using-mathlib4-as-a-dependency>.
- [MP71] Robert McNaughton and Seymour A. Papert. *Counter-Free Automata*. MIT Press, 1971.

- [MU21] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. *Automated Deduction – CADE 28*. Springer, 2021, pp. 625–635. DOI: [10.1007/978-3-030-79876-5_37](https://doi.org/10.1007/978-3-030-79876-5_37).
- [Ope26a] OpenAI. *Custom Instructions with AGENTS.md*. Accessed 22 July 2026. 2026. URL: <https://learn.openai.com/docs/agent-configuration/agents-md>.
- [Ope26b] OpenAI. *Planar Point Sets with Many Unit Distances*. Includes the original problem prompt and a statement on automated generation and layered review. 2026. URL: <https://cdn.openai.com/pdf/74c24085-19b0-4534-9c90-465b8e29ad73/unit-distance-proof.pdf>.
- [PST92] Jean-Éric Pin, Howard Straubing, and Denis Thérien. “Some Results on the Generalized Star-Height Problem”. *Information and Computation* 101.2 (1992), pp. 219–250. DOI: [10.1016/0890-5401\(92\)90063-L](https://doi.org/10.1016/0890-5401(92)90063-L). URL: <https://hal.science/hal-00019978>.
- [PZ17] Thomas Place and Marc Zeitoun. *Generic Results for Concatenation Hierarchies*. 2017. arXiv: [1710.04313](https://arxiv.org/abs/1710.04313) [cs.FL]. URL: <https://arxiv.org/abs/1710.04313>.
- [Sch65] Marcel-Paul Schützenberger. “On Finite Monoids Having Only Trivial Subgroups”. *Information and Control* 8.2 (1965), pp. 190–194. DOI: [10.1016/S0019-9958\(65\)90108-7](https://doi.org/10.1016/S0019-9958(65)90108-7).
- [The20] The mathlib Community. “The Lean Mathematical Library”. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs* (2020), pp. 367–381. DOI: [10.1145/3372885.3373824](https://doi.org/10.1145/3372885.3373824).